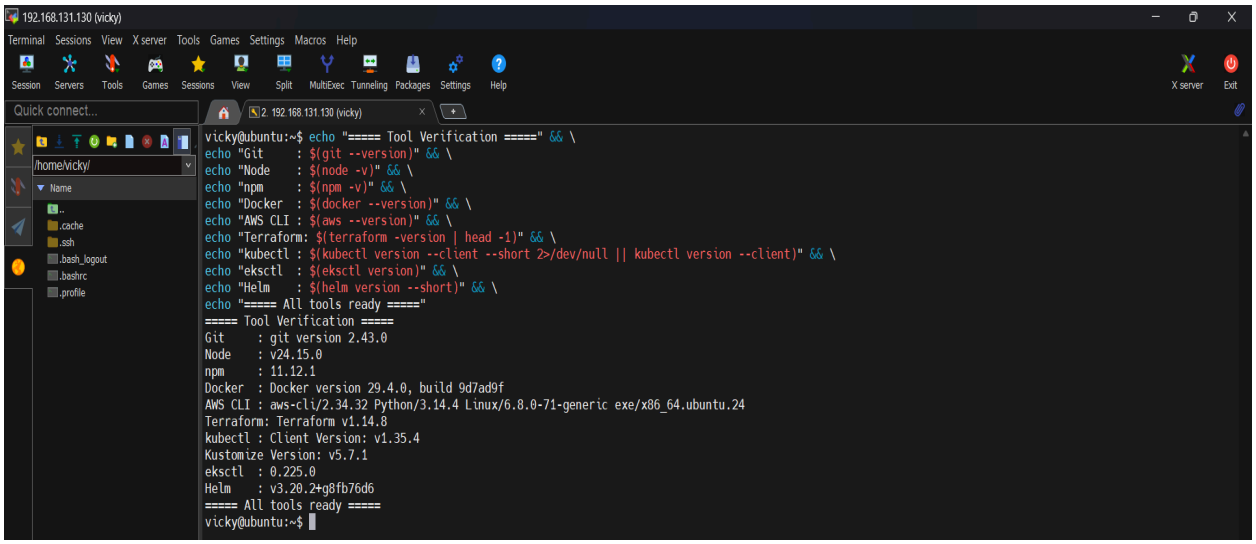
TRENDSTORE APPLICATION

Github Repo :
https://github.com/wikkics/trendstore-app
Loadbalancer ARN :
http://ad1639eccd79245b2a0eaac1838ee8c5-481365027.us-east-1.elb.amazonaws.com

Tools & Technologies Installed :

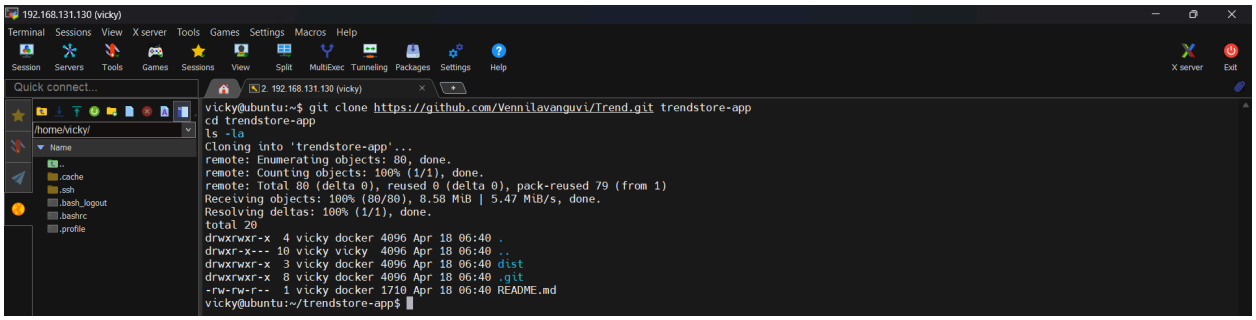
Tools / Technology	Purpose
Git	Used for version control, cloning repositories, and pushing code to GitHub
Node.js	Runs the React application locally (default port: 3000)
npm	Installs and manages project dependencies
Docker	Builds and runs containerized applications
AWS CLI	Enables interaction with AWS services from the terminal
Terraform	Provisions and manages AWS infrastructure as code
kubectl	Command-line tool to interact with Kubernetes clusters
eksctl	Creates and manages Amazon EKS (Kubernetes) clusters
Helm	Deploys applications like Prometheus and Grafana on Kubernetes



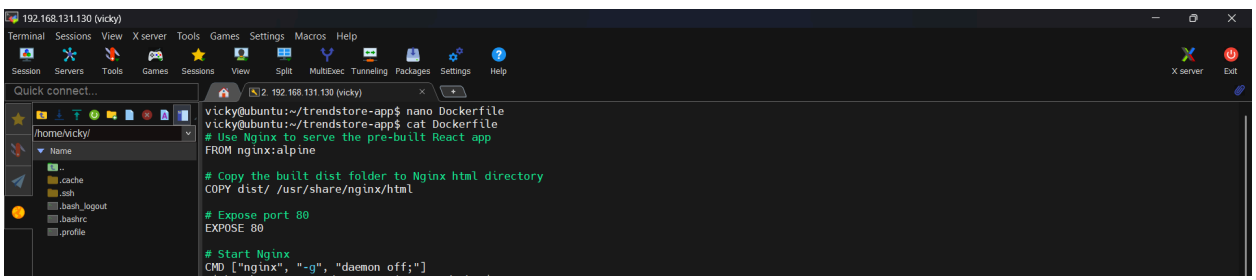
Phase 1 — App Setup & Docker

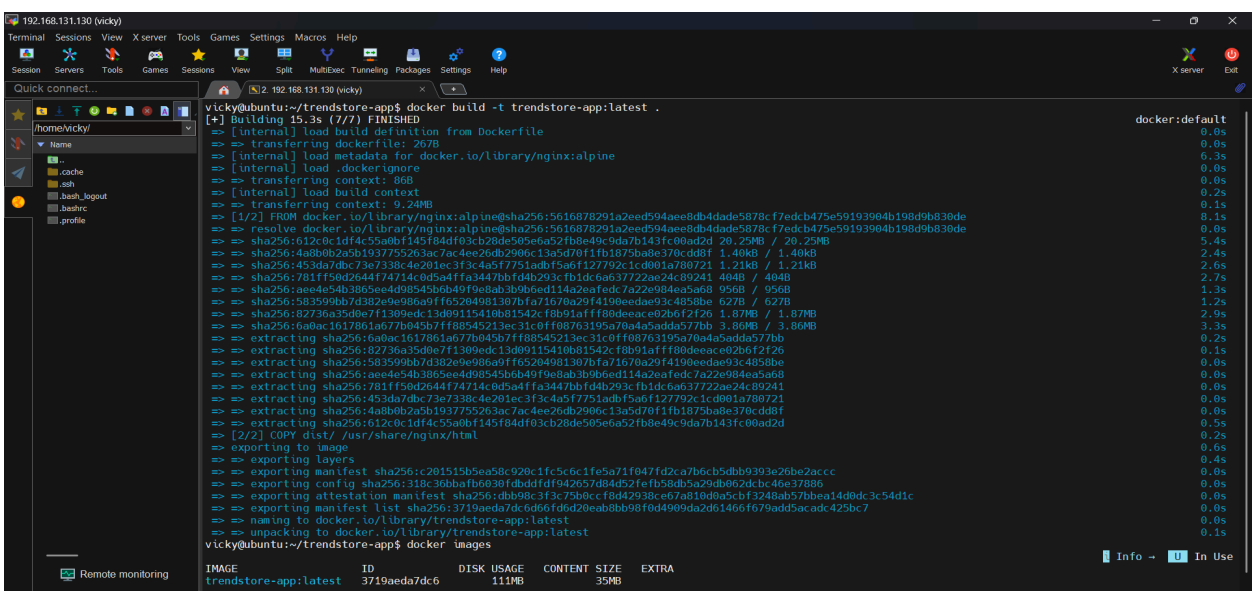
In this phase, I cloned the React application repository, I created a Dockerfile, built a Docker image, and verified the containerized application works correctly.

Clone the Github Repository



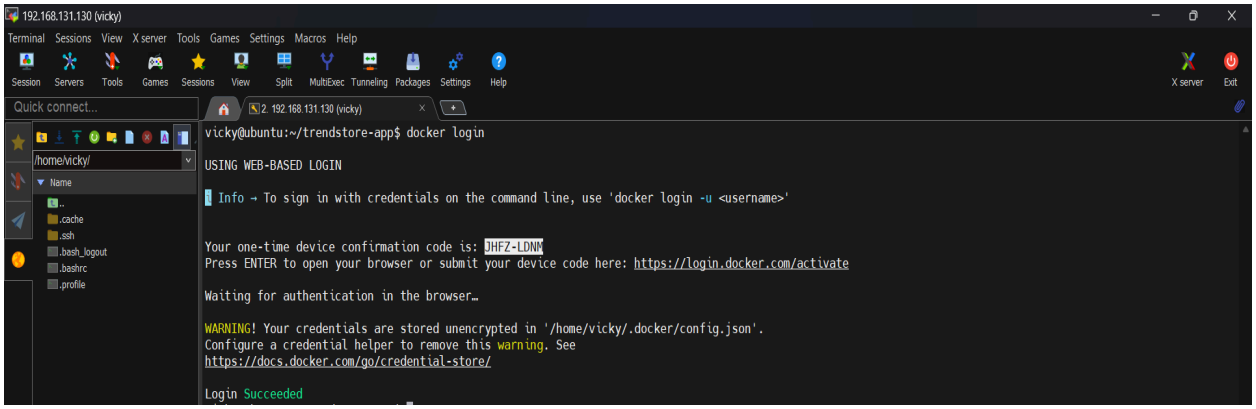
I created a `Dockerfile` in the project root directory. Since the application was already built and available in the `dist/` folder, I used Nginx to directly serve the static files.I built the Docker image using the Dockerfile I created and tagged it as `trend-app:latest`.

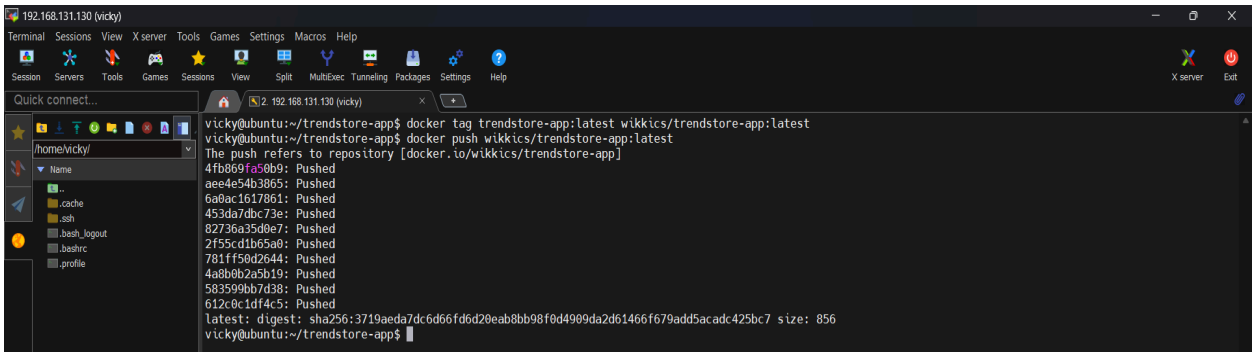


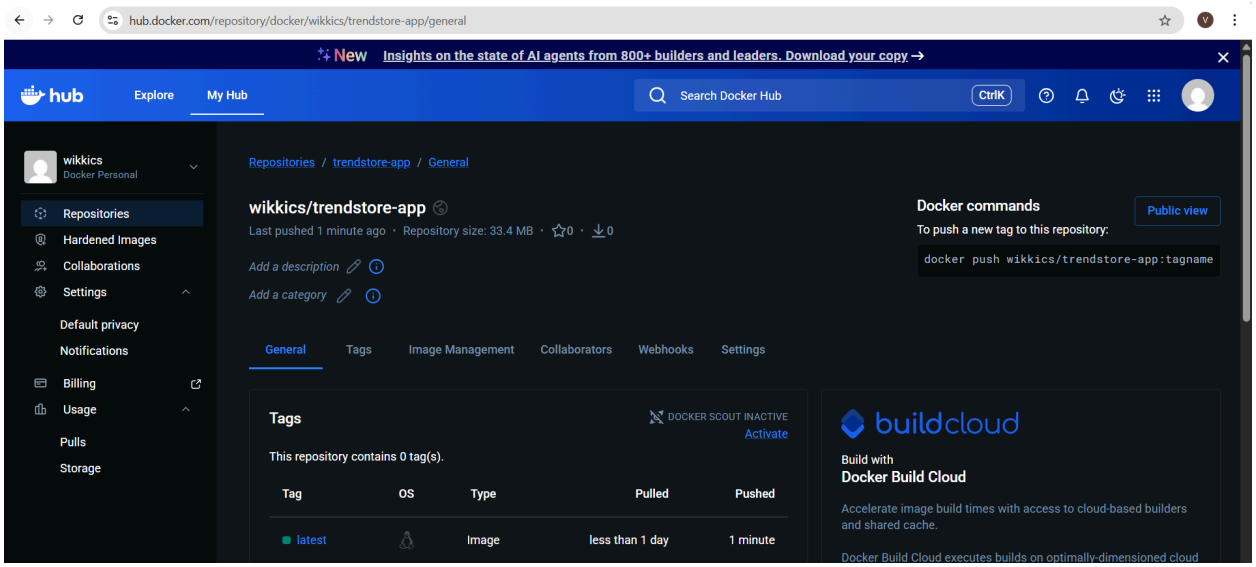


Phase 2 — DockerHub Registry

I logged in to DockerHub from terminal using browser-based authentication. I created a public repository named `trendstore-app` on DockerHub portal. Then I tagged the local Docker image with my DockerHub username and pushed it successfully.



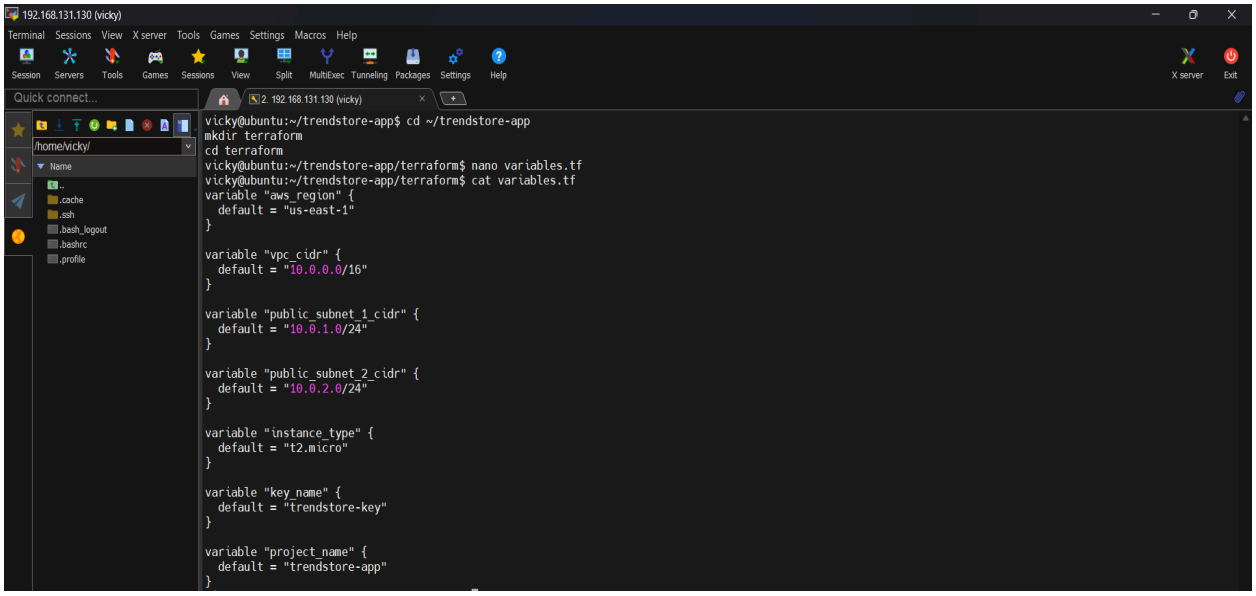


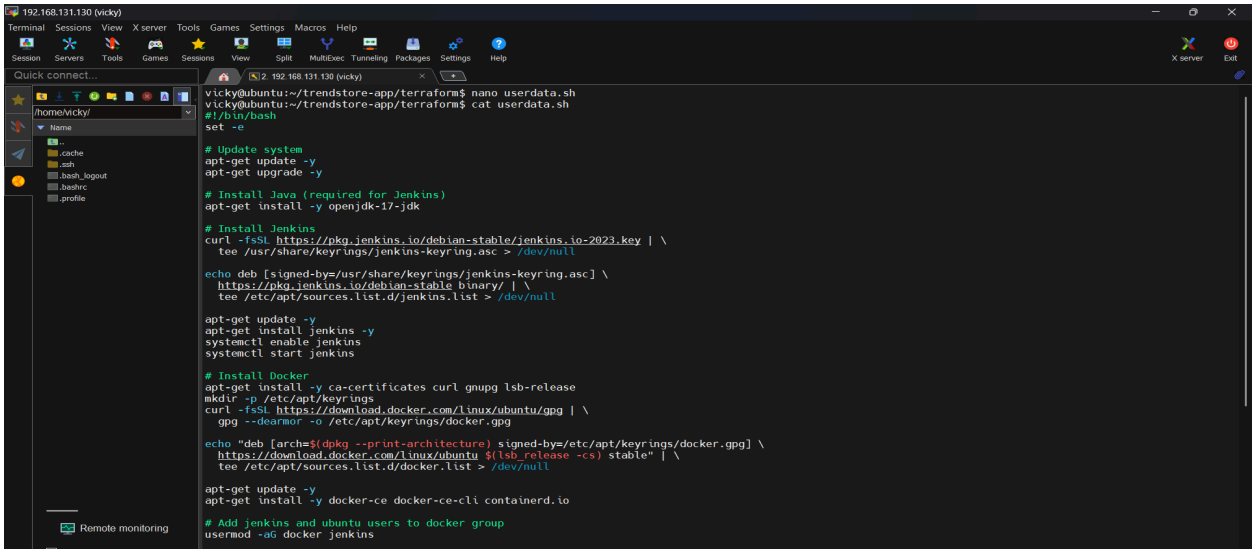


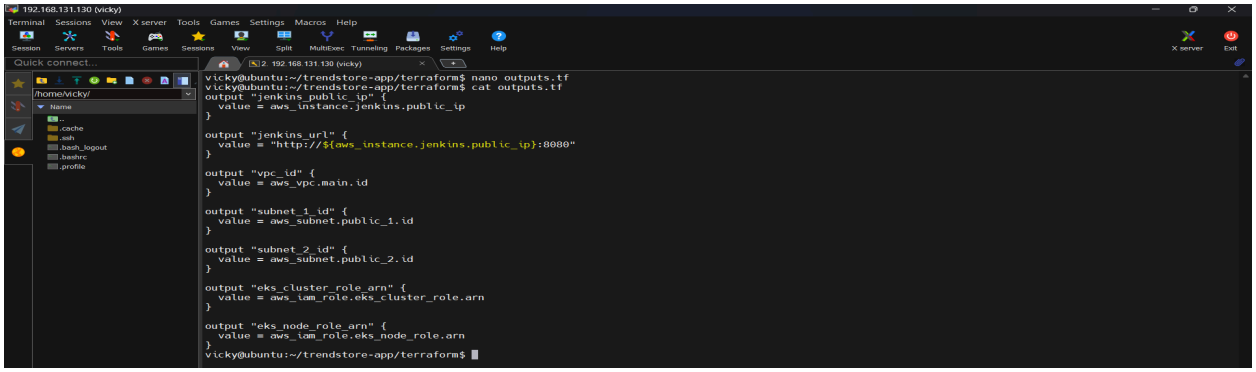
Phase 3 — Terraform Infrastructure

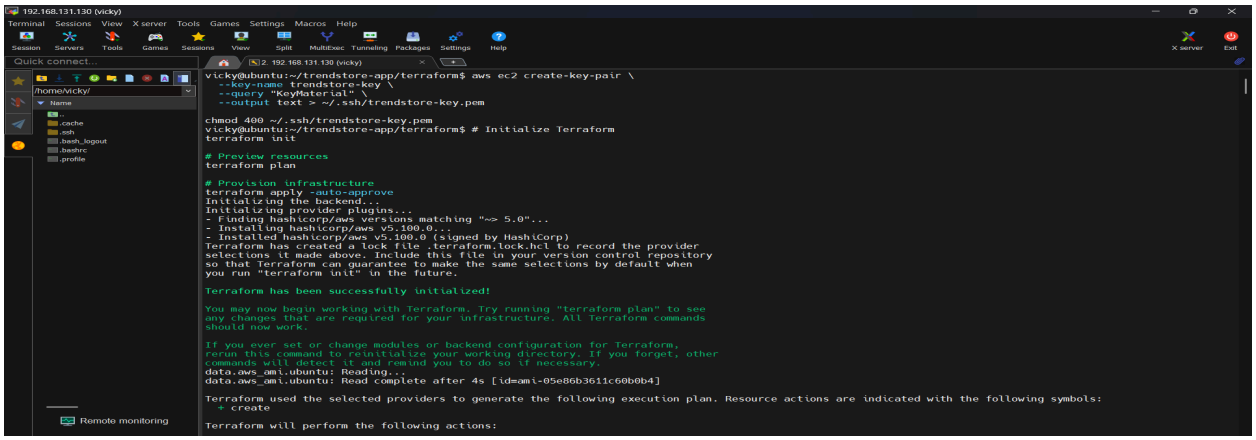
I created a `terraform/` directory inside the project and wrote four files — `variables.tf`, `main.tf`, `outputs.tf`, and `userdata.sh`. I used `us-east-1` region and `t3.micro` instance type for free tier eligibility.

The infrastructure defined included VPC with CIDR `10.0.0.0/16`, two public subnets in availability zones `us-east-1a` and `us-east-1b`, internet gateway, route tables, security groups allowing ports 22, 80 and 8080, IAM roles for EKS cluster and node group, and a `t3.micro` EC2 instance for Jenkins with 20GB storage. The `userdata.sh` script automatically installed Jenkins, Docker, kubectl and AWS CLI on the EC2 instance at startup.



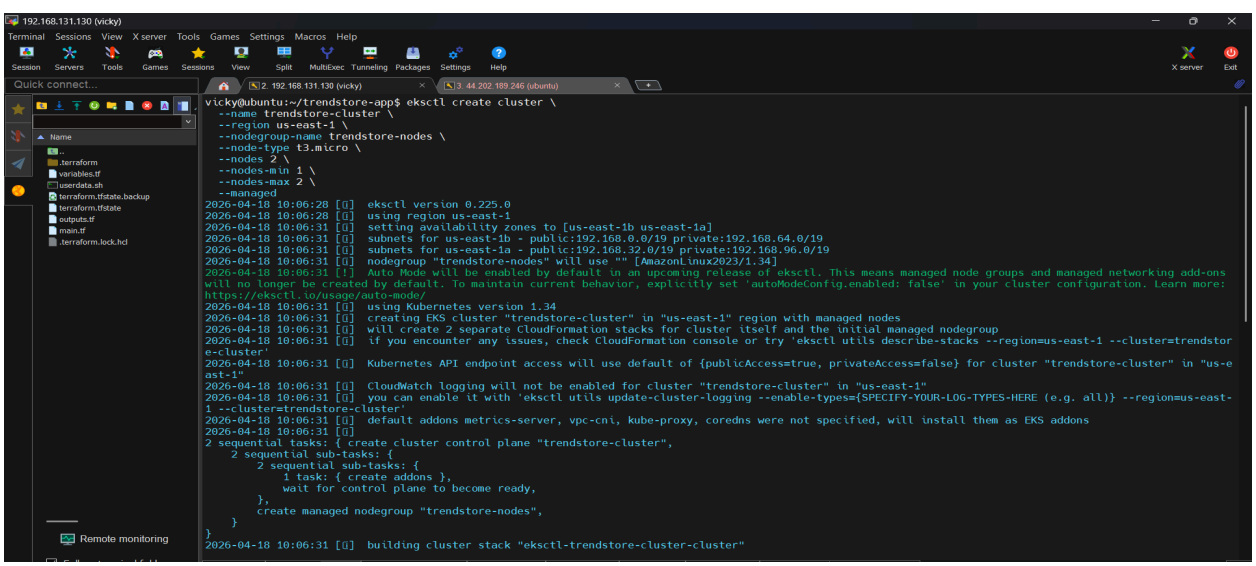


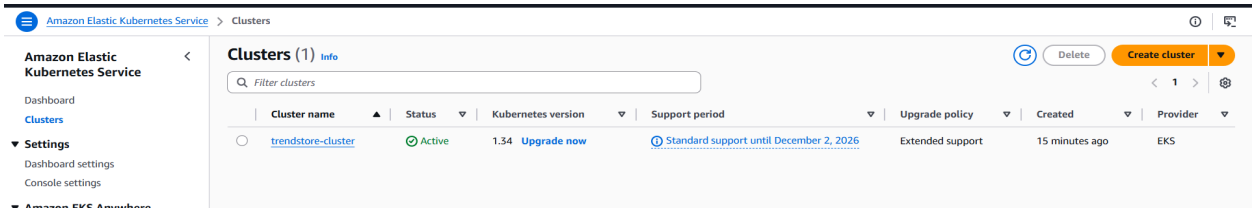


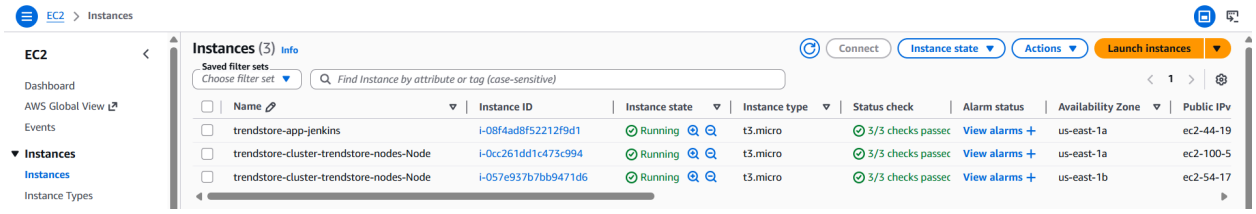


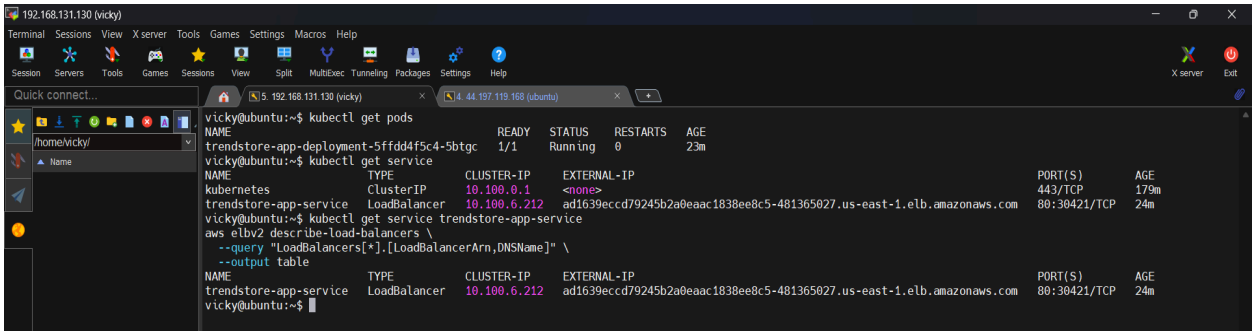
Phase 4 — Kubernetes on AWS EKS

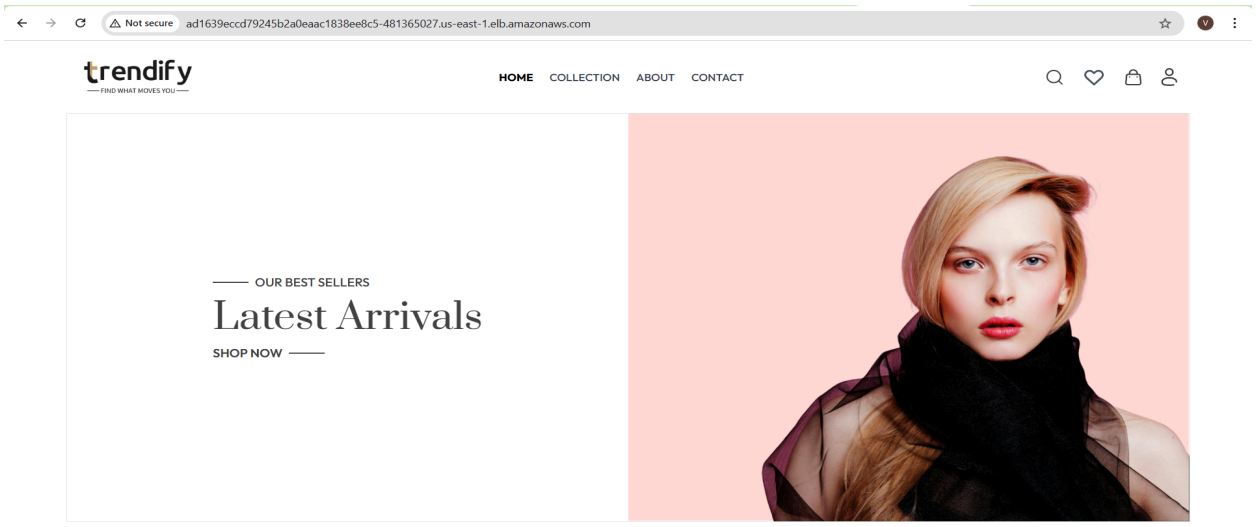
Created an AWS EKS cluster, wrote Kubernetes deployment and service YAML files, and deployed the application exposed via a LoadBalancer I created the EKS cluster using eksctl with 2 managed nodes. Initially used `t3.micro` nodes but hit pod limit issues since `t3.micro` allows only 4 pods per node and system pods consumed all slots. I upgraded to `t3.small` nodes which support 11 pods per node.




Amazon Elastic Kubernetes Service > Clusters
Amazon Elastic Kubernetes Service
Dashboard
Clusters
Settings
Dashboard settings
Console settings
Clusters (1) Info
Delete
Create cluster
Filter clusters
Cluster name
Status
Kubernetes version
Support period
Upgrade policy
Created
Provider
trendstore-cluster
Active
1.34 Upgrade now
Standard support until December 2, 2026
Extended support
15 minutes ago
EKS

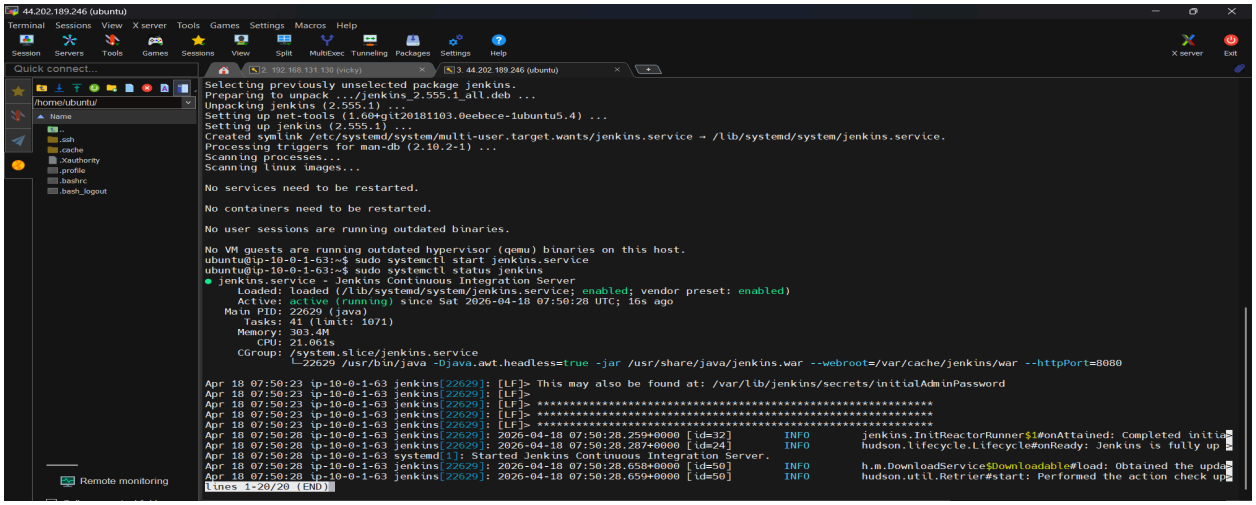

EC2 > Instances
EC2
Dashboard
AWS Global View
Events
Instances
Instances
Instance Types
Instances (3) Info
Connect
Instance state
Actions
Launch instances
Saved filter sets
Choose filter set
Find Instance by attribute or tag (case-sensitive)
Name
Instance ID
Instance state
Instance type
Status check
Alarm status
Availability Zone
Public IPv
trendstore-app-jenkins
i-08f4ad8f52212f9d1
Running
t3.micro
3/3 checks passed
View alarms
us-east-1a
ec2-44-19
trendstore-cluster-trendstore-nodes-Node
i-0cc261dd1c473c994
Running
t3.micro
3/3 checks passed
View alarms
us-east-1a
ec2-100-5
trendstore-cluster-trendstore-nodes-Node
i-057e937b7bb9471d6
Running
t3.micro
3/3 checks passed
View alarms
us-east-1b
ec2-54-17

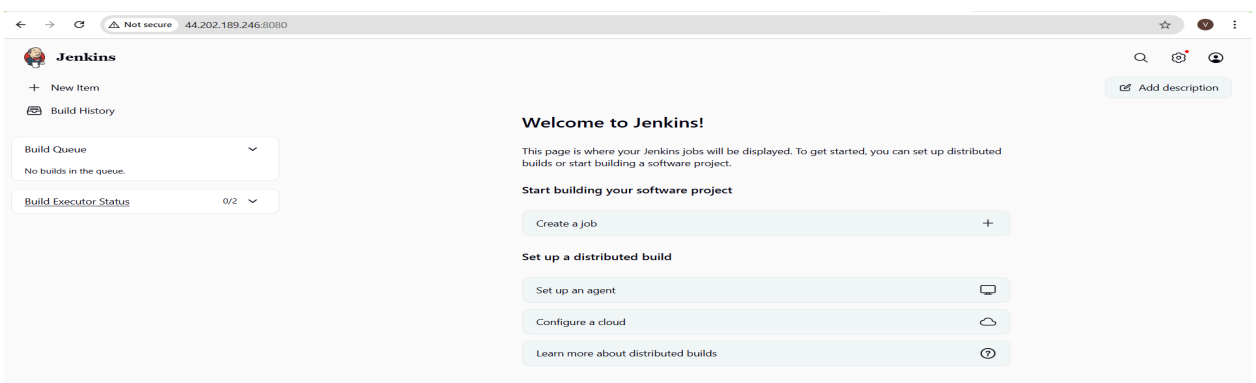

192.168.131.130 (vicky)
Terminal Sessions View X server Tools Games Settings Macros Help
Session Servers Tools Games Sessions View Split MultiExec Tunneling Packages Settings Help
X server Exit
Quick connect...
/home/vicky/
Name
5. 192.168.131.130 (vicky)
4. 44.197.119.168 (ubuntu)
vicky@ubuntu:~$ kubectl get pods
NAME READY STATUS RESTARTS AGE
trendstore-app-deployment-5ffdd4f5c4-5btgc 1/1 Running 0 23m
vicky@ubuntu:~$ kubectl get service
NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE
kubernetes ClusterIP 10.100.0.1 <none> 443/TCP 179m
trendstore-app-service LoadBalancer 10.100.6.212 ad1639eccd79245b2a0eaac1838ee8c5-481365027.us-east-1.elb.amazonaws.com 80:30421/TCP 24m
vicky@ubuntu:~$ kubectl get service trendstore-app-service
aws elbv2 describe-load-balancers \
--query "LoadBalancers[*].[LoadBalancerArn,DNSName]" \
--output table
NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE
trendstore-app-service LoadBalancer 10.100.6.212 ad1639eccd79245b2a0eaac1838ee8c5-481365027.us-east-1.elb.amazonaws.com 80:30421/TCP 24m
vicky@ubuntu:~$

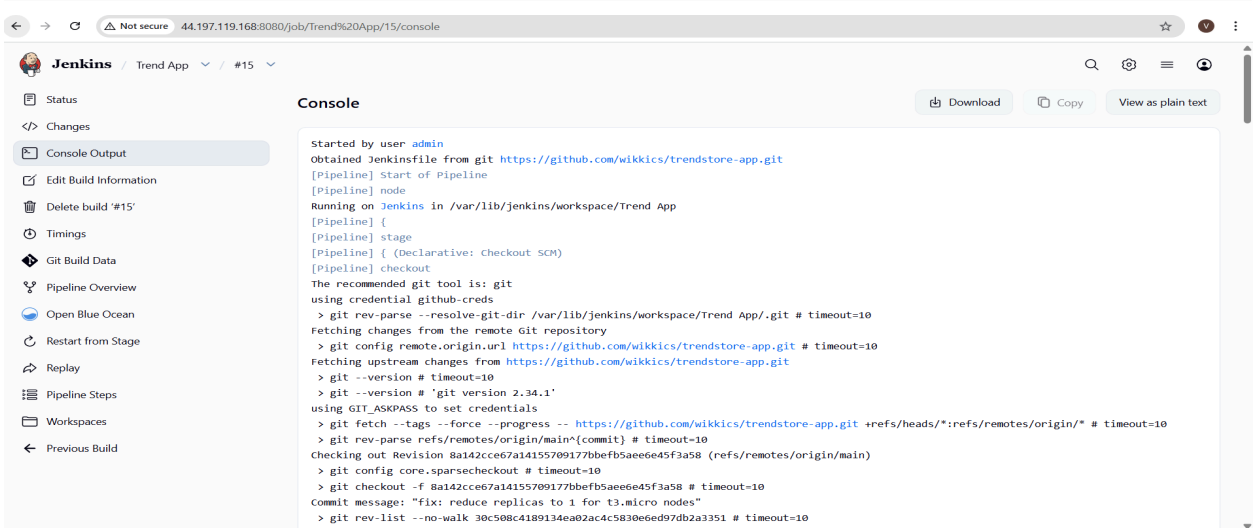

Not secure
ad1639eccd79245b2a0eaac1838ee8c5-481365027.us-east-1.elb.amazonaws.com
trendify
FIND WHAT MOVES YOU
HOME COLLECTION ABOUT CONTACT
OUR BEST SELLERS
Latest Arrivals
SHOP NOW

Phase 5 — Jenkins CI/CD Pipeline

Set up Jenkins on EC2, integrated with GitHub, and created a declarative pipeline that automatically builds the Docker image, pushes to DockerHub, and deploys to EKS on every code commit.I completed the Jenkins initial setup at `http://44.202.189.246:8080` and installed the following plugins: Docker Pipeline, Git Plugin, GitHub Integration, Kubernetes CLI, Credentials Binding and Pipeline







Not secure 44.197.119.168:8080/job/Trend%20App/15/console

Jenkins / Trend App / #15

```
Commit message: "fix: reduce replicas to 1 for t3.micro nodes"
 > git rev-list --no-walk 30c508c4189134ea02ac4c5830e6ed97db2a3351 # timeout=10
[Pipeline] }
[Pipeline] // stage
[Pipeline] withEnv
[Pipeline] {
[Pipeline] withCredentials
Masking supported pattern matches of $DOCKERHUB_CREDENTIALS or $DOCKERHUB_CREDENTIALS_PSW or $AWS_ACCESS_KEY_ID or $AWS_SECRET_ACCESS_KEY
[Pipeline] {
[Pipeline] withEnv
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Checkout)
[Pipeline] echo
Cloning repository...
[Pipeline] checkout
The recommended git tool is: git
using credential github-creds
 > git rev-parse --resolve-git-dir /var/lib/jenkins/workspace/Trend App/.git # timeout=10
Fetching changes from the remote Git repository
 > git config remote.origin.url https://github.com/wikkics/trendstore-app.git # timeout=10
Fetching upstream changes from https://github.com/wikkics/trendstore-app.git
 > git --version # timeout=10
 > git --version # 'git version 2.34.1'
using GIT_ASKPASS to set credentials
 > git fetch --tags --force --progress -- https://github.com/wikkics/trendstore-app.git +refs/heads/*:refs/remotes/origin/* # timeout=10
 > git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision 8a142cce67a14155709177bbefb5aee6e45f3a58 (refs/remotes/origin/main)
```

Not secure 44.197.119.168:8080/job/Trend%20App/15/console

Jenkins / Trend App / #15

```
 > git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision 8a142cce67a14155709177bbefb5aee6e45f3a58 (refs/remotes/origin/main)
 > git config core.sparsecheckout # timeout=10
 > git checkout -f 8a142cce67a14155709177bbefb5aee6e45f3a58 # timeout=10
Commit message: "fix: reduce replicas to 1 for t3.micro nodes"
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Build Docker Image)
[Pipeline] echo
Building Docker image...
[Pipeline] sh
+ docker build -t wikkics/trendstore-app:15 .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  9.309MB

Step 1/4 : FROM nginx:alpine
 ---> 5616878291a2
Step 2/4 : COPY dist/ /usr/share/nginx/html
 ---> Using cache
 ---> 091563820d63
Step 3/4 : EXPOSE 80
 ---> Using cache
 ---> d03a5a0bdffc
Step 4/4 : CMD ["nginx", "-g", "daemon off;"]
```

Not secure 44.197.119.168:8080/job/Trend%20App/15/console

Jenkins / Trend App / #15

```
+ kubectl get pods
NAME                                      READY   STATUS    RESTARTS   AGE
trendstore-app-deployment-5ffdd4f5c4-5btgc   1/1     Running   0          13s
+ kubectl get service trendstore-app-service
NAME                    TYPE           CLUSTER-IP     EXTERNAL-IP                                                              PORT(S)
AGE
trendstore-app-service   LoadBalancer   10.100.6.212   ad1639eccd79245b2a0eaac1838ee8c5-481365027.us-east-1.elb.amazonaws.com   80:30421/TCP
17s
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] sh
+ docker logout
Removing login credentials for https://index.docker.io/v1/
[Pipeline] echo
Pipeline completed successfully. Application deployed!
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withCredentials
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Not secure 44.197.119.168:8080/job/Trend%20App/multi-pipeline-graph/

Jenkins / Trend App / Stages

Stages

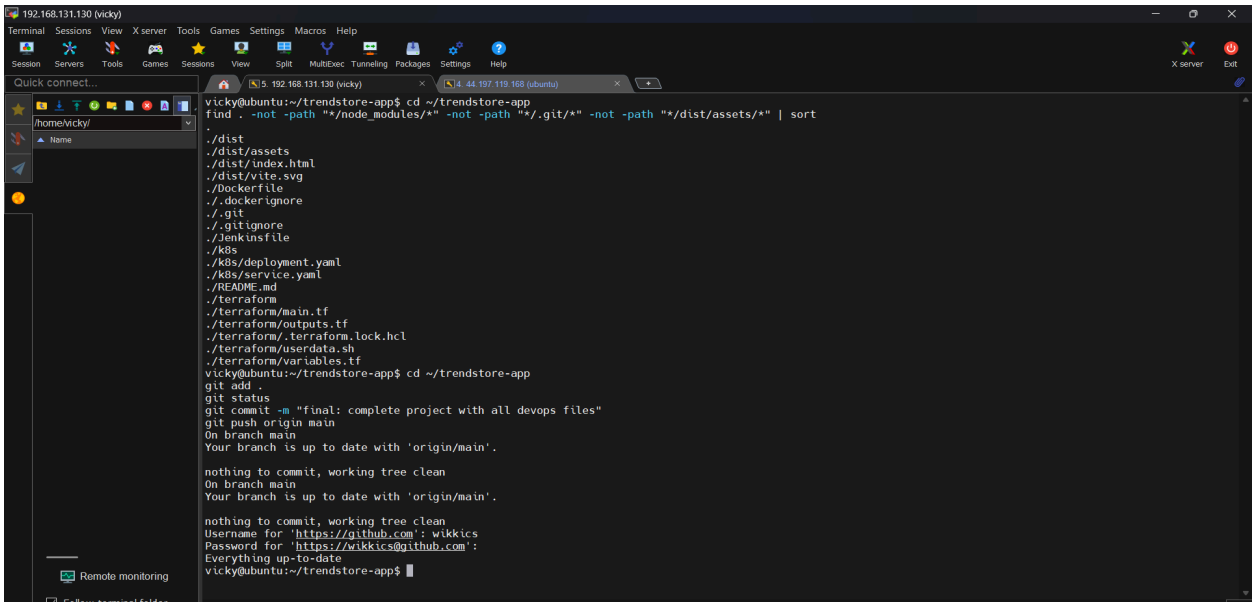
Build

18 April 2026

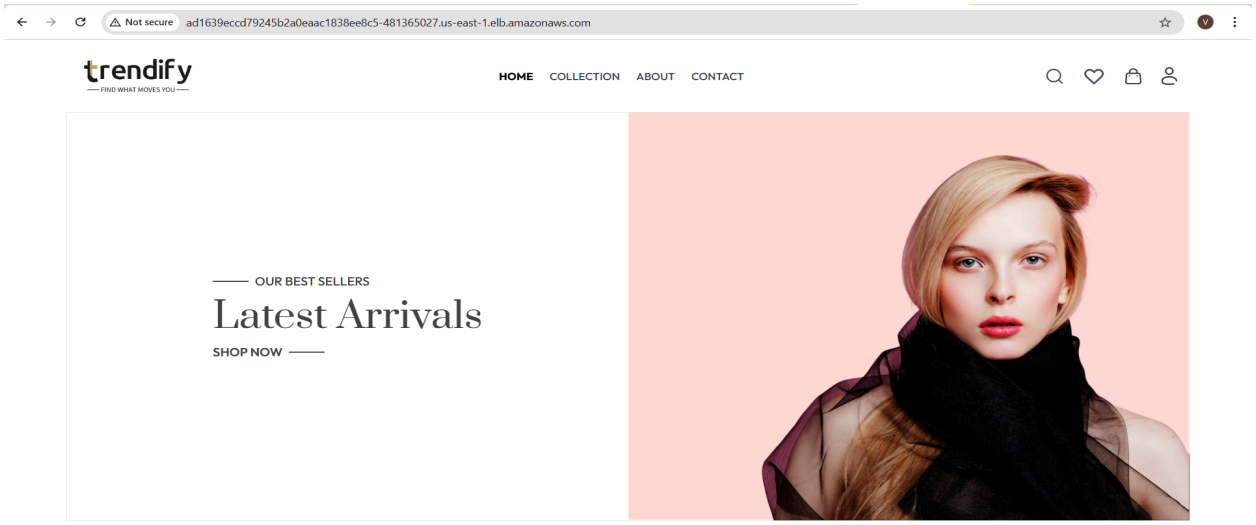
#15
18:17 - 32s - 1 change

Phase 6 — Version Control:

Organized the complete project repository with proper ignore files and pushed the final clean codebase to GitHub.

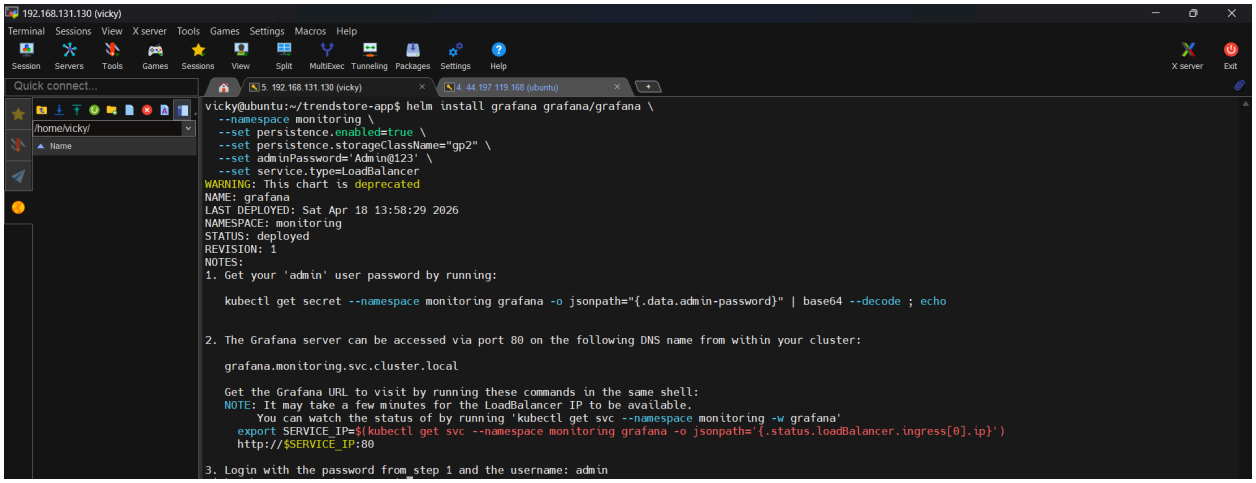


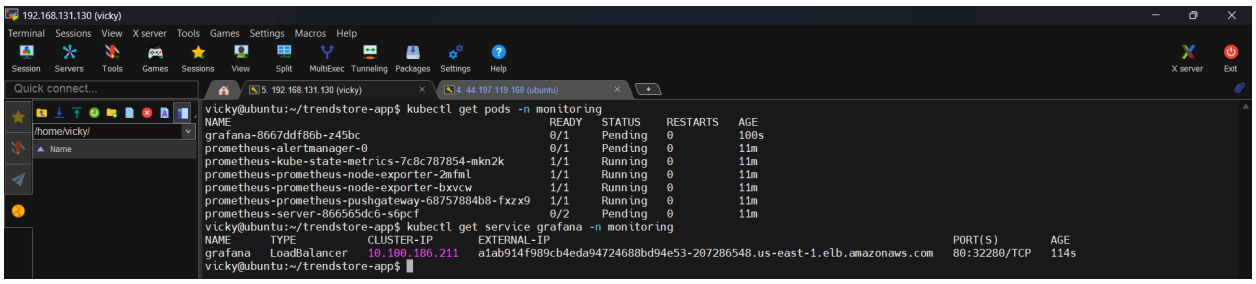
Website Access Using ARN :

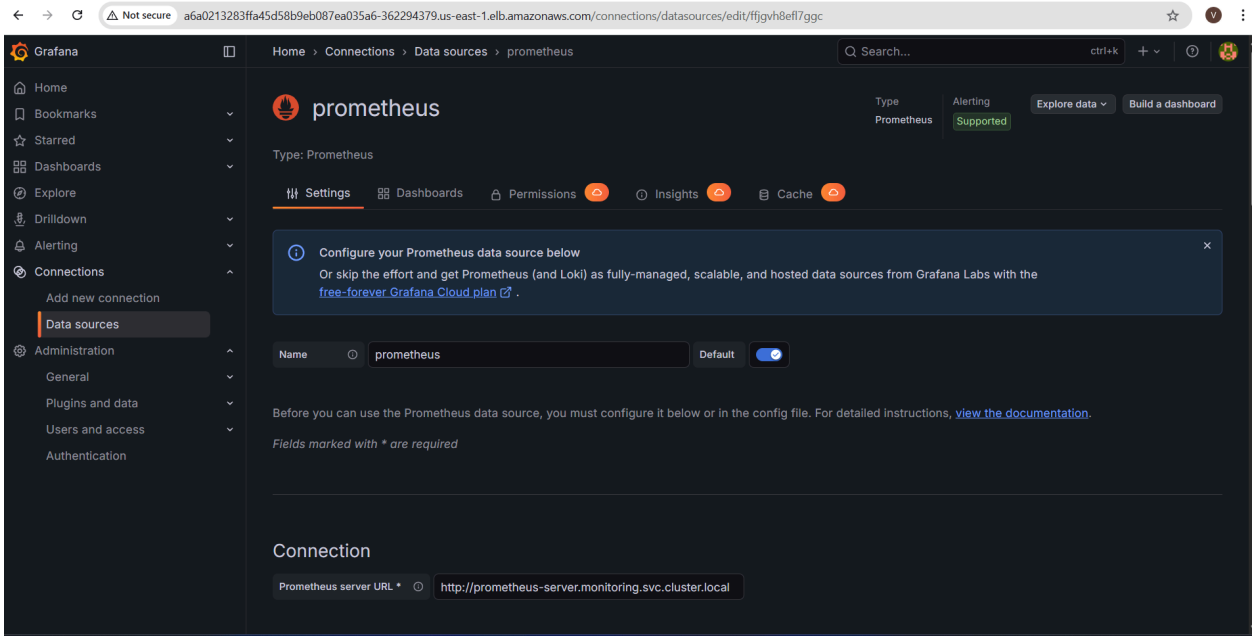


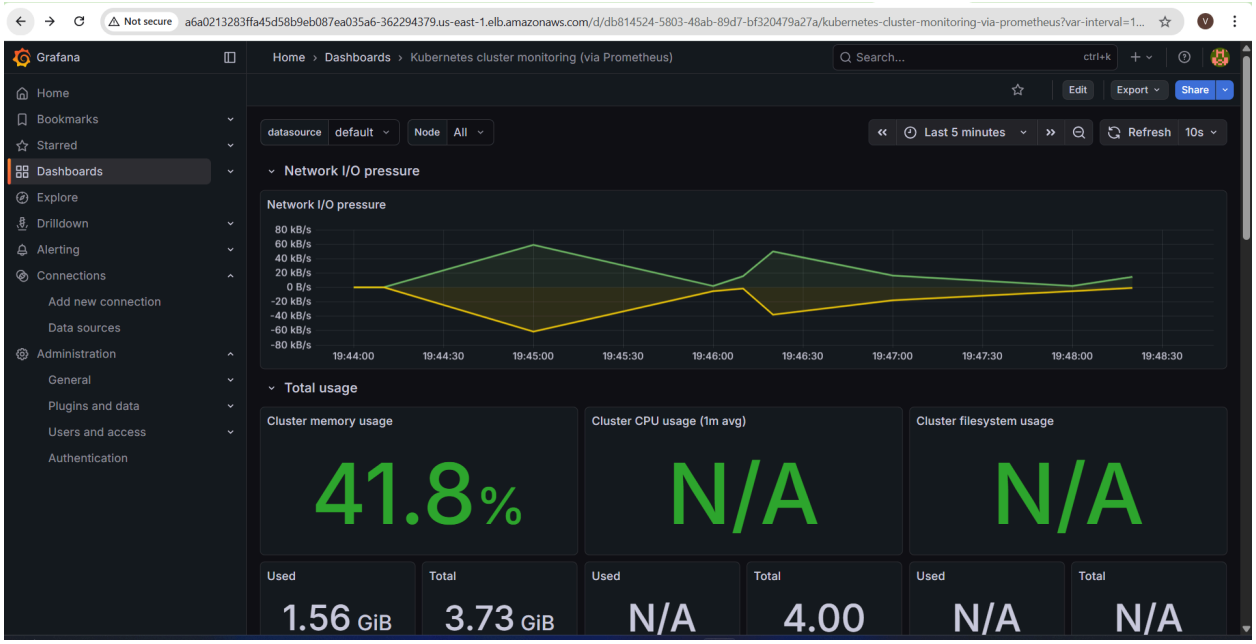
Phase 7 — Monitoring with Prometheus and Grafana

Set up Prometheus and Grafana on the EKS cluster to monitor application health, pod status, CPU and memory usage with real-time dashboards.I added the Helm repositories for Prometheus and Grafana, created a monitoring namespace, and installed both tools on the EKS cluster.








Not secure
a6a0213283ffa45d58b9eb087ea035a6-362294379.us-east-1.elb.amazonaws.com/d/db814524-5803-48ab-89d7-bf320479a27a/kubernetes-cluster-monitoring-via-prometheus?var-interval=1...
Grafana
Home
Bookmarks
Starred
Dashboards
Explore
Drilldown
Alerting
Connections
Add new connection
Data sources
Administration
General
Plugins and data
Users and access
Authentication
Home › Dashboards › Kubernetes cluster monitoring (via Prometheus)
Search...
ctrl+k
Edit
Export
Share
datasource
default
Node
All
Last 5 minutes
Refresh
10s
Network I/O pressure
Network I/O pressure
80 kB/s
60 kB/s
40 kB/s
20 kB/s
0 B/s
-20 kB/s
-40 kB/s
-60 kB/s
-80 kB/s
19:44:00
19:44:30
19:45:00
19:45:30
19:46:00
19:46:30
19:47:00
19:47:30
19:48:00
19:48:30
Total usage
Cluster memory usage
41.8%
Cluster CPU usage (1m avg)
N/A
Cluster filesystem usage
N/A
Used
1.56 GiB
Total
3.73 GiB
Used
N/A
Total
4.00
Used
N/A
Total
N/A